

ПРОЦЕСС ПОСТРОЕНИЯ ДЕРЕВА РЕШЕНИЙ

Этап 1 — Определение цели и сбор данных

Цель — нужно четко сформулировать задачу, которую нужно решить, например оценить вероятность определенного события.

Сбор данных — подобрать качественные данные, подходящие для анализа. Это могут быть файлы, базы данных, датасеты или потоки данных, которые содержат признаки, влияющие на конечное решение.

Пример: допустим, цель — предсказать, купит ли покупатель продукт. Данные будут включать демографические, психологические и поведенческие характеристики покупателей.

Этап 2 — Выбор признаков и разбиение данных

На этом этапе нужно выявить наиболее важные признаки для **разбиения данных**. Это разделение датасета на подмножества на основе выбранного атрибута (признака).

На каждом узле выбираем критерий, который наилучшим образом разделяет данные на однородные группы.

Пример:

Шаг 1 — корневой узел (возраст)

Начинаем с признака «Возраст», который лучше всего делит покупателей на группы:

- молодые (до 30 лет): 80% не покупают, 20% покупают;
- взрослые (30 лет и старше): 40% не покупают, 60% покупают.

Шаг 2 — узел 1 (молодые покупатели)

Следующий признак — «Доход». Разделим молодых покупателей на две группы:

- низкий доход: 90% не покупают, 10% покупают;
- высокий доход: 50% не покупают, 50% покупают.

Шаг 3 — узел 2 (взрослые покупатели)

Для взрослых покупателей используем признак «Место проживания»:

- проживание в городе: 30% не покупают, 70% покупают;
- проживание в пригороде: 60% не покупают, 40% покупают.

Процесс разбиения продолжается до выполнения условий остановки, таких как минимальный размер узла или достижение заданной глубины дерева.

Критерии разбиения данных

Рассмотрим, как именно оценивать правильность выбора признака для разбиения данных. Для этого есть следующие критерии.

Энтропия (неопределенность). Показывает, насколько группы данных являются определенными и однородными после разбиения. Чем ниже энтропия, тем лучше разбиение.

$$H(S) = - \sum_{i=1}^c p_i(P_i)$$

где:

- $H(S)$ — энтропия множества S ;
- c — количество классов в данных;
- p_i — вероятность того, что случайно выбранный объект принадлежит классу i .

Индекс Джини — это еще один показатель однородности групп данных. Чем ниже индекс Джини, тем больше данных относится к одной категории. Поэтому выбирают такие условия, чтобы индекс был минимальным.

$$G(S) = 1 - \sum_{i=1}^c p_i^2$$

где:

- $G(S)$ — индекс Джини для множества S ;
- c — количество классов в данных;
- p_i — вероятность того, что случайно выбранный объект принадлежит классу i .

Алгоритмы построения дерева

Существуют алгоритмы построения деревьев решений, которые используют разные подходы для разбиения данных.

ID3 (Iterative Dichotomiser 3)

Выбирает признак, который максимально уменьшает энтропию. Для этого он использует информационную выгоду, которая показывает, насколько хорошо признак делит данные на группы. Применение: подходит для небольших наборов данных и задач классификации, но плохо работает с непрерывными (числовыми) признаками и склонен к переобучению.

Этап 3 — Условия остановки

1. Достигнута определенная глубина дерева
2. Количество данных в узле становится слишком маленьким
3. Не удастся улучшить разделение данных (например, не уменьшает энтропию или индекс Джини), алгоритм может остановиться.

Этап 4 — Отсечение ветвей

Существует два вида.

Предварительное отсечение (Pre-Pruning)

С его помощью можно ограничить рост дерева до того, как оно станет слишком сложным. Основные методы:

- **Максимальная глубина.** Устанавливают ограничения на максимальную глубину дерева.
- **Минимум единиц данных на лист.** Устанавливают минимальное количество единиц данных в каждом листе.
- **Минимум единиц данных для разбиения.** Указывают минимальное число единиц данных для деления.
- **Максимальное количество признаков.** Вводят ограничение на максимальное число признаков для разбиения.

Последующее отсечение (Post-Pruning)

Это удаление узлов или ветвей после завершения роста дерева. Основные методы:

- **Отсечение для сокращения стоимости и сложности.** Этот метод позволяет отсечь неэффективные ветви и повысить эффективность модели.
- **Отсечение излишних ветвей.** Удаление ветвей, которые не влияют на общую точность.

Этап 5 — Оценка и тестирование дерева

После построения дерево тестируют на новых данных, чтобы убедиться, что оно правильно классифицирует или предсказывает результаты.

Обучение решающих деревьев, L классов

Обучение решающих деревьев Рассмотрим задачу распознавания с классами $K_1, \dots, K_L$. Обучение производится по обучающей выборке $\tilde{S}_t$ и включает в себя поиск оптимальных пороговых параметров или оптимальных дихотомических разбиений для признаков $X_1, \dots, X_n$. При этом поиск производится исходя из требования снижения среднего индекса неоднородности в выборках, порождаемых искомым дихотомическим разбиением обучающей выборки $\tilde{S}_t$.

Индекс неоднородности вычисляется для произвольной выборки $\tilde{S}$, содержащей объекты из классов $K_1, \dots, K_L$. При этом используется несколько видов индексов, включая:

- энтропийный индекс неоднородности,
- индекс Джини,
- индекс ошибочной классификации.

Энтропийный индекс неоднородности вычисляется по формуле

$$\gamma_e(\tilde{S}) = - \sum_{i=1}^L P_i \ln P_i, \quad (1)$$

где P_i - доля объектов класса K_i в выборке $\tilde{S}$. При этом принимается, что $0 \ln(0) = 0$. Наибольшее значение $\gamma_e(\tilde{S})$ принимает при равенстве долей классов. Наименьшее значение $\gamma_e(\tilde{S})$ достигается при принадлежности всех объектов одному классу.

Индекс Джини вычисляется по формуле

$$\gamma_g(\tilde{S}) = 1 - \sum_{i=1}^L P_i^2. \quad (2)$$

Индекс ошибочной классификации вычисляется по формуле

$$\gamma_m(\tilde{S}) = 1 - \max_{1, \dots, L} (P_i). \quad (3)$$

Нетрудно понять, что индексы (2) и (3) также достигают минимального значения при принадлежности всех объектов обучающей выборке одному классу.

Предположим, что в методе обучения используется индекс неоднородности γ_* . Для оценки эффективности разбиения обучающей выборки $\tilde{S}_t$ на непересекающиеся подвыборки $\tilde{S}_t^l$ и $\tilde{S}_t^r$ используется уменьшение среднего индекса неоднородности в $\tilde{S}_t^l$ и $\tilde{S}_t^r$ по отношению к $\tilde{S}_t$

Данное уменьшение вычисляется по формуле

$$\Delta(\gamma_*, \tilde{S}_t) = \gamma_*(\tilde{S}_t) - P_l \gamma_*(\tilde{S}_t^l) - P_r \gamma_*(\tilde{S}_t^r),$$

где P_l и P_r являются долями $\tilde{S}_t^l$ и $\tilde{S}_t^r$ в выборке $\tilde{S}_t$. На первом этапе обучения бинарного решающего дерева ищется оптимальный предикат соответствующий корневой вершине. С этой целью оптимальные разбиения строятся для каждого из признаков из набора $X_1, \dots, X_n$. Выбирается признак $X_{i_{max}}$ с максимальным значением индекса $\Delta(\gamma_*, \tilde{S}_t)$. Подвыборки $\tilde{S}_t^l$ и $\tilde{S}_t^r$, задаваемые оптимальным предикатом для $X_{i_{max}}$ оцениваются с помощью критерия остановки. В качестве критерия остановки может быть использован простейший критерий достижения полной однородности по одному из классов. В случае, если какая-нибудь из выборок $\tilde{S}_t^*$ удовлетворяет критерию остановки, то соответствующая вершина дерева объявляется концевой и для неё вычисляется метка класса. В случае, если выборка $\tilde{S}_t^*$ не удовлетворяет критерию остановки, то формируется новая внутренняя вершина, для которой процесс построения дерева продолжается

Применение метода на практике

Рассмотрим, как работает дерево решений, на примере задачи.

Задача

Банку нужен алгоритм, по которому он будет определять, одобрять клиенту кредит или нет. Критериями будут такие факторы, как возраст, доход, кредитная история и статус занятости.

На основании данных о предыдущих клиентах можно будет построить дерево решений для определения платежеспособности следующих клиентов.

Этапы построения упрощенного дерева решений

1. Сбор данных

Собираем данные о клиентах, включающие следующие параметры:

- **Возраст:** до 25 лет, 25–40 лет, более 40 лет.
- **Доход:** низкий, средний, высокий.
- **Кредитная история:** положительная или отрицательная.
- **Статус занятости:** трудоустроен или нет.

2. Построение дерева решений

- **Корневой узел.** Начинаем с выбора критерия с наибольшей информационной выгодой. В данном случае это может быть кредитная история.
- **Первый уровень разбиения**
Если кредитная история **положительная**, идем по ветке к узлу «Высокая вероятность одобрения».
Если кредитная история **отрицательная**, идем по ветке к узлу, где проверяются другие факторы.

3. Дальнейшее разбиение узлов

- **Узел 1:** если кредитная история положительная, проверяем доход.
Лист 1: если доход высокий, итоговое решение — **одобрено**.
Лист 2: если доход средний, также итог — **одобрено** (с некоторыми оговорками).
Лист 3: если доход низкий, итог — **отказано**.
- **Узел 2:** если кредитная история отрицательная, проверяем возраст.
Лист 4: если возраст старше 40 лет и доход высокий, итог — **одобрено**.
Лист 5: если возраст старше 40 лет и доход низкий, итог — **отказано**.
Лист 6: если возраст младше 25 лет, независимо от дохода, итог — **отказано**.
Лист 7: если возраст от 25 до 40 лет и доход высокий, итог — **одобрено**.
Лист 8: если возраст от 25 до 40 лет и доход низкий, итог — **отказано**.

Итоги

- Деревья решений — это метод, который помогает принять решение и систематизировать данные.
- Дерево состоит из корневого узла (root node), внутренних узлов (internal node), ветвей (branch) и листьев (leaf).
- В корневом узле задают начальный вопрос. Затем по веткам ответов последовательно переходят к следующим узлам или листьям. Узлы «задают» вопрос, а листья выдают результат.
- К одному результату (листу) могут приводить несколько ветвей.
- Деревья решений можно строить вручную с помощью электронных таблиц и платформ для визуализации. Также деревья создают с помощью моделей машинного обучения.
- Чтобы создавать обученные модели деревьев решений, нужно разбираться в машинном обучении и знать языки программирования. У Python и R есть специальные библиотеки для построения деревьев решений.
- Процесс построения дерева решений состоит из нескольких этапов. В него входят определение цели, сбор данных, выбор признаков, разбиение данных, остановка построения, отсечение ветвей и тестирование дерева.

Точность распознавания рассчитывается как отношение объектов, правильно классифицированных в процессе обучения, к общему количеству объектов набора данных, которые принимали участие в обучении.

Ошибка рассчитывается как отношение объектов, неправильно классифицированных в процессе обучения, к общему количеству объектов набора данных, которые принимали участие в обучении.

Древовидные алгоритмы: ID3, C4.5, C5.0 и CART

Что представляют собой различные алгоритмы деревьев решений и чем они отличаются друг от друга? Какой из них реализован в scikit-learn?

ID3 (Iterative Dichotomiser 3) был разработан в 1986 году Россом Куинланом. Алгоритм создает многоходовое дерево, находя для каждого узла (т. е. жадным образом) категориальный признак, который *даст наибольший прирост информации* для категориальных целей. Деревья растут до максимального размера, а затем обычно применяется шаг обрезки, чтобы улучшить способность дерева обобщаться на невидимые данные.

C4.5 является преемником ID3 и снимает ограничение на то, что признаки должны быть категориальными, динамически определяя дискретный атрибут (основанный на числовых переменных), который разбивает непрерывное значение признака на дискретный набор интервалов. C4.5 преобразует обученные деревья (т. е. результаты работы алгоритма ID3) в наборы правил “если - то”. Затем оценивается точность каждого правила, чтобы определить порядок их применения. Обрезка производится путем удаления предварительного условия правила, если без него точность правила повышается.

C5.0 - это последняя версия Куинлана, выпущенная под проприетарной лицензией. Она использует меньше памяти и строит меньшие наборы правил, чем C4.5, при этом являясь более точной.

CART (Classification and Regression Trees) очень похож на C4.5, но отличается от него тем, что поддерживает числовые целевые переменные (регрессия) и не вычисляет наборы правил. CART строит бинарные деревья, используя признак и порог, которые дают наибольший прирост информации в каждом узле.